

# CPSC 304 Project Cover Page

Milestone # : 2

Date : 22-October-2025

Group Number : 73

| Name               | UBC Student # | UBC CSID | Email                   |
|--------------------|---------------|----------|-------------------------|
| Jaden Victor Majid | 94916335      | n6q1z    | majid.jaden@gmail.com   |
| Trevor Kozek       | 29541778      | d7h3b    | tkozek@student.ubc.ca   |
| Raymond Lee        | 36028942      | t7z4t    | lraymond@student.ubc.ca |

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

## MILESTONE 2

### SUMMARY OF PROJECT

This project is a centralized database for a theme park designed to integrate ticketing, hotel reservation systems, loyalty programs, and ride operations. By consolidating all customer and operational data into one platform, it enables staff and management to efficiently monitor guest activity, apply rate modifiers, validate ride access, and track maintenance.

### ER DIAGRAM

**\*Please find the updated ER diagram at the end of this document.\***

As per suggestions provided by our project mentor in milestone 1, we have altered the relationship between loyalty member and seasons pass to be one-to-one and associated the RateModifier entity with the Booking entity as opposed to Hotel entity to centralize the rate modifier logic. In addition to this, we have added a participation constraint to SeasonPass when participating in the HoldsA relation, as we believe it best reflects real-world application where every season pass belongs to a loyalty member.

We have also added a number of attributes that we believe are meaningful and add to our number of functional dependencies for the normalization exercise. The changes are noted below:

- *Cost* added to Booking entity
- *ValidHours* was added to Ticket entity
- *StreetAddress, City, Province* added to Hotel entity
- *Category* added to RoomType entity
- *NumberOfWorkers* added to MaintenanceRecord

We have also renamed a number of attributes to avoid replication of names. These changes are noted below:

- *Name* changed to *CustomerName* (in Customer entity)
- *Name* changed to *RideName* (in Ride entity)
- *ValidityEnd* changed to *ValidUntil* (in Ticket entity)
- *UUID* changed to *HotelBookingID* (in HotelStay entity)
- *Name* changed to *RoomName* (in RoomType entity)
- *ValidityStart* changed to *SeasonStart* (in SeasonPass entity)
- *ValidityEnd* changed to *SeasonEnd* (in SeasonPass entity)
- *Date* changed to *MaintenanceDate* (in MaintenanceRecord entity)

In our feedback for milestone 1, we were also asked to discuss the differences between our theme park data management system with retail management. While there is a very small overlap between these topics as they both involve customers, the majority of our system deals with topics and entities that are well beyond the scope of a retail management system. For example, theme parks (ie. Disneyland) often have on-site hotels and accommodation which are typically not offered in a retail management scenario. Our system also tracks ticketing for rides

which require maintenance to be logged and recorded, whereas a retail management system would typically deal with totally different concerns such as efficient inventory management. From our perspective, the majority of our ER diagram is not relevant to retail management and therefore we believe it constitutes a significant enough difference to justify designing a totally different system.

## **RELATIONAL SCHEMA**

### **Entities**

MaintenanceRecord(**RideName**: VARCHAR, **RecordID**: INTEGER, MaintenancePerformed: VARCHAR, MaintenanceDate: DATE, NumberOfWorkers: INTEGER)

- Merged with HoldsA
- Primary Key: RideName, RecordID
- Foreign Key: RideName

*\*This colour & underline means partial key as google sheets does not allow dotted underlines.*

Ride(**RideName**: VARCHAR, Capacity: INTEGER, InstallDate: DATE)

- Primary Key: RideName

Ticket(**TicketID**: INTEGER, **BookingNumber**: INTEGER, ValidFrom: DATE, ValidUntil: DATE, RemainingUses: INTEGER, ValidHours: INTEGER)

- merged w/ HasTickets
- Primary Key: TicketID
- Foreign Key: BookingNumber

FastPassTicket(**TicketID**: INTEGER, NumberOfRides: INTEGER, ValidHours: INTEGER)

TicketID is foreign key

- Primary Key: TicketID
- Foreign Key: TicketID

Customer(**CustomerID**: INTEGER, CustomerName: VARCHAR, Sex: VARCHAR, DOB: DATE)

- Primary Key: CustomerID

Guest(**CustomerID**: INTEGER, DateOfVisit: DATE)

- Primary Key: CustomerID
- Foreign Key: CustomerID

LoyaltyMember(**CustomerID**: INTEGER, LoyaltyID: INTEGER, Points: INTEGER)

- Merged with HoldsA
- Primary Key: CustomerID
- Candidate Key: LoyaltyID
- Foreign Key: CustomerID

Booking(BookingNumber: INTEGER, BookingDate: DATE, Cost: INTEGER, **CustomerID**: INTEGER)

- merged with Reserves
- Primary Key: BookingNumber
- Foreign Key: CustomerID (NOT NULL)

HotelStay(HotelBookingID: INTEGER, NumGuests: INTEGER, CheckInDate: DATE, CheckOutDate: DATE, **HotelName**: VARCHAR, **RoomName**: VARCHAR, **BookingNumber**: INTEGER)

- Merged with BookedAt, AtRoomType, HasHotelStay
- Primary Key: HotelBookingID
- Foreign Key: HotelName (NOT NULL), RoomName (NOT NULL), BookingNumber (NOT NULL)

RoomType(RoomName: VARCHAR, MaxGuests: INTEGER, BaseRate: FLOAT, Category: VARCHAR)

- Primary Key: RoomName

Hotel(HotelName: VARCHAR, StreetAddress: VARCHAR, City: VARCHAR, Province: VARCHAR, PostalCode: VARCHAR)

- Primary Key: HotelName

RateModifier(RateCode: INTEGER, Modifier: FLOAT)

- Primary key: RateCode

SeasonPass(UUID: INTEGER, SeasonPassLevel: VARCHAR, SeasonStart: DATE, SeasonEnd: DATE, **LoyaltyID**: INTEGER)

- Merged with HoldsA
- Primary Key: UUID
- Foreign Key: LoyaltyID (NOT NULL, UNIQUE)

## Relations

ForRide(RideName: VARCHAR, **TicketID**: INTEGER)

- Primary Key: RideName, TicketID
- Foreign Key: RideName, TicketID

OffersRoomType(**HotelName**: VARCHAR, **RoomName**: VARCHAR, Quantity: INTEGER)

- Primary Key: HotelName, RoomName
- Foreign Key: HotelName, RoomName

BookedWithRateModifier(**RateCode**: INTEGER,

**BookingNumber**: INTEGER)

- Primary Key: RateCode, BookingNumber
- Foreign Key: RateCode, BookingNumber

SeasonPassSpecial(**UUID**: INTEGER,

**RateCode**: INTEGER)

- Primary Key: UUID, RateCode
- Foreign Key: UUID, RateCode

## **FUNCTIONAL DEPENDENCIES (FDS)**

Maintenance Record

- RideName, RecordID -> MaintenancePerformed, MaintenanceDate, Capacity, InstallDate, NumberOfWorkers
- MaintenancePerformed -> NumberOfWorkers

Ride

- RideName -> Capacity, InstallDate

Ticket

- TicketID -> ValidFrom, ValidUntil, RemainingUses, ValidHours
- ValidFrom, ValidHours -> ValidUntil

FastPassTicket

- TicketID -> NumberOfRides, ValidFrom, ValidUntil, RemainingUses, ValidHours

Customer

- CustomerID -> Sex, CustomerName, DOB

Guest

- CustomerID -> DateOfVisit, Sex, CustomerName, DOB

LoyaltyMember

- CustomerID -> LoyaltyID, Points, UUID, SeasonPassLevel, SeasonStart, SeasonEnd, Sex, CustomerName, DOB
- LoyaltyID -> CustomerID, Points, UUID, SeasonPassLevel, SeasonStart, SeasonEnd, Sex, CustomerName, DOB

Booking

- BookingNumber -> Cost, BookingDate, CustomerID, CustomerName, DOB, Sex
- BookingDate -> Cost

#### SeasonPassSpecial

- UUID, RateCode -> Modifier, SeasonPassLevel, SeasonStart, SeasonEnd

#### BookedWithRateModifier

- RateCode, BookingNumber -> Modifier, BookingDate, Cost

#### OffersRoomType

- HotelName, RoomName -> Quantity, MaxGuests, BaseRate, Category, StreetAddress, City, Province, PostalCode

#### Hotel

- PostalCode -> City, Province

#### ForRide

- RideName, TicketID -> InstallDate, Capacity, ValidFrom, ValidUntil

#### SeasonPass

- UUID -> SeasonPassLevel, SeasonStart, SeasonEnd, LoyaltyID, Points, Sex, CustomerID, CustomerName, DOB
- SeasonPassLevel, SeasonStart -> SeasonEnd

#### RateModifier

- RateCode -> Modifier

#### RoomType

- RoomName -> MaxGuests, BaseRate, Category
- Category -> BaseRate

#### HotelStay

- HotelBookingID -> CheckInDate, CheckOutDate, NumGuests, HotelName, StreetAddress, City, Province, PostalCode, RoomName, MaxGuests, BaseRate, BookingDate, BookingNumber, Cost

### **NORMALIZATION**

\*These relations were normalized to be in BCNF\*

MaintenanceRecord1(MaintenancePerformed: VARCHAR, NumberOfWorkers: INTEGER)

- Primary Key: MaintenancePerformed

MaintenanceRecord2(**RideName**: VARCHAR, RecordID: INTEGER, MaintenancePerformed: VARCHAR, MaintenanceDate: DATE)

- Primary Key: RideName, RecordID
- Foreign Key: RideName

*\*This colour & underline means partial key as google sheets does not allow dotted underlines.*

Ride(RideName: VARCHAR, Capacity: INTEGER, InstallDate: DATE)

- Primary Key: RideName

Ticket1(ValidFrom: DATE, ValidUntil: DATE, ValidHours: INTEGER)

- Primary Key: ValidFrom, ValidHours

Ticket2(TicketID: INTEGER, **BookingNumber**: INTEGER, **ValidFrom**: DATE, RemainingUses: INTEGER, **ValidHours**: INTEGER)

- Primary Key: TicketID
- Foreign Key: BookingNumber, ValidFrom, ValidHours

FastPassTicket(**TicketID**: INTEGER, NumberOfRides: INTEGER, ValidHours: INTEGER)

TicketID is foreign key

- Primary Key: TicketID
- Foreign Key: TicketID

Customer(CustomerID: INTEGER, CustomerName: VARCHAR, Sex: VARCHAR, DOB: DATE)

- Primary Key: CustomerID

Guest(**CustomerID**: INTEGER, DateOfVisit: DATE)

- Primary Key: CustomerID
- Foreign Key: CustomerID

LoyaltyMember(**CustomerID**: INTEGER, LoyaltyID: INTEGER, Points: INTEGER)

- Primary Key: CustomerID
- Candidate Key: LoyaltyID
- Foreign Key: CustomerID

Booking1(BookingDate: DATE, Cost: INTEGER)

- Primary Key: BookingDate

Booking2(BookingNumber: INTEGER, BookingDate: DATE, **CustomerID**: INTEGER)

- Primary Key: BookingNumber
- Foreign Key: CustomerID (NOT NULL)

HotelStay(HotelBookingID: INTEGER, NumGuests: INTEGER, CheckInDate: DATE, CheckOutDate: DATE, **HotelName**: VARCHAR, **RoomName**: VARCHAR, **BookingNumber**: INTEGER)

- Merged with BookedAt, AtRoomType, HasHotelStay
- Primary Key: HotelBookingID
- Foreign Key: HotelName (NOT NULL), RoomName (NOT NULL), BookingNumber (NOT NULL)

RoomType1(BaseRate: FLOAT, Category: VARCHAR)

- Primary Key: Category

RoomType2(RoomName: VARCHAR, MaxGuests: INTEGER, Category: VARCHAR)

- Primary Key: RoomName

Hotel1(City: VARCHAR, Province: VARCHAR, PostalCode: VARCHAR)

- Primary Key: PostalCode

Hotel2(HotelName: VARCHAR, StreetAddress: VARCHAR, PostalCode: VARCHAR)

- Primary Key: HotelName

RateModifier(RateCode: INTEGER, Modifier: FLOAT)

- Primary key: RateCode

SeasonPass1(SeasonPassLevel: VARCHAR, SeasonStart: DATE, SeasonEnd: DATE,)

- Primary Key: SeasonPassLevel, SeasonStart

SeasonPass2(UUID: INTEGER, SeasonPassLevel: VARCHAR, SeasonStart: DATE, **LoyaltyID**: INTEGER)

- Primary Key: UUID
- Foreign Key: LoyaltyID (NOT NULL, UNIQUE)

ForRide(**RideName**: VARCHAR, **TicketID**: INTEGER)

- Primary Key: RideName, TicketID
- Foreign Key: RideName, TicketID

OffersRoomType(**HotelName**: VARCHAR, **RoomName**: VARCHAR, Quantity: INTEGER)

- Primary Key: HotelName, RoomName
- Foreign Key: HotelName, RoomName

BookedWithRateModifier(**RateCode**: INTEGER, **BookingNumber**: INTEGER)



- Primary Key: RateCode, BookingNumber
- Foreign Key: RateCode, BookingNumber

SeasonPassSpecial(**UUID**: INTEGER,  
**RateCode**: INTEGER)

- Primary Key: UUID, RateCode
- Foreign Key: UUID, RateCode

### **SQL DDL**

Included as checkpoint2.sql

Create Table & Insert statements are included in the sql file.

Context around foreign key handling (ie. ON DELETE / ON UPDATE) have been added as comments in the sql file.

### **USE OF AI TOOLS IN MILESTONE 2**

No AI tools were used to assist in creating our submission for Milestone 2

